

Documentation on Multiplexer

Brisa Ramirez

October 2025

1 Contents

1.1 Introduction

.....2

1.2 What is a Multiplexer used for?

- Data Routing.....4
- Microprocessors.....4
- Memory Management.....4
- Signal Processing.....4

1.3 System overview

- Overview on Parameterization.....(4 and 5)

1.4 Design Objective

.....5

1.5 Behavioral Modeling

.....5

1.6 Architecture

- Matrix Operation6

1.7 Parameter Configuration

.....6

1.8 Module Descriptions

- Core Arithmetic Modules.....6
- project_1.v (Full adder).....7
- sub_controller (subtraction Logic).....7
- signed_adder_subtract.v.....7
- ripple_carry_16bit.v8

1.9	Matrix Multiplication	
•	structural_matrix_multiplication.....	8
1.10	Test bench Architecture	
•	signed_adder_subtractor_tb.v.....	8
•	signed_multiplier_tb.v.....	9
•	structural_matrix_tb.v.....	9
1.11	Key Design Decisions	
•	Signed Arithmetic Handling.....	9
•	16-bit precision.....	10
•	Structural Implementation.....	10
•	Error Handling.....	10
1.12	Performance Characteristics	
•	Performance requirements.....	10
•	Timing Characteristics.....	10
1.13	Conclusion	
		10

2 Introduction

This project demonstrates the construction of digital arithmetic system in Verilog. The system is implemented using a hierarchical bottom-up design approach. That begins with a basic combinational logic components and progressing towards larger and more complex arithmetic modules. A Multiplexer is a combinational circuit that selects one input line from multiple data inputs and forwards it to a single output line based on the value of a selection signal optimizing data transmission and resource utilization. This project presents the design and implementation of a 16-bit multiplexer in Verilog HDL using two different approaches. The documentation includes the methods used, data flow, and the data modeling techniques. In this design, we focus on a 2:1 Multiplexer, which selects between input bits a and b based on selected line sel.

3 Used for

3.1 Multiplexer can be used for

- Data Routing: Enables the transmission of multiple signals over a signals over a single channel, enhancing bandwidth efficiency.
- Microprocessors: Used to select between different data sources or instructions, facilitating efficient processing.
- Memory Management: Helps managing memory connections, reducing the number of physical connections needed.
- Signal Processing: Utilized in analog and digital signal processing to manage multiple input signals effectively.

4 System Desing Overview

The system operates on 16-bit signed values, defined centrally in the header file, which is Definitions.vh.

This parameterization ensures that every module in the project can be easily scaled to different bit-widths without requiring changes to the internal logic of each module. By controlling the data width and the interval results width through shared definitions, the system remains maintainable and flexible.

The design consists of several major stages. First, a Half adder is used to provide the fundamental bit-level sum and carry operations. This forms the basis of the full Adder, constructed structurally using two half adders and OR gate. Multiple full adders are chained to create a Ripple-Carry adder, which performs multi-bit addition. A subtraction controller is then integrated, allowing the ripple-carry adder to support both signed addition and subtraction using two's complement logic. These arithmetic modules are subsequently used to form

unsigned and signed multipliers, which are utilized in the final 3 by 3 matrix multiplication module.

- Half-adder: Serves as the primitive sum-and-carry generator.
- Full-adder: Implemented using two Half adder, handles single-bit addition with carry.
- Subtraction Controller: Converts subtraction operations to two's complement form.
- Signed Adder-Subtractor: Incorporates both addition and subtraction capabilities with overflow detection.
- Unsigned Multiplier: Extends multiplication to signed numbers via sign control.
- The 3x3 Matrix Multiplication: Unit uses nine multipliers and six adders structurally.
At every stage, module boundaries are respected, ensuring clean signal flow, consistent bit-widths, and correct operand handling.

5 Design Objective

The goal is to create a Verilog 16-bit input based on a single-bit that select a signal. Having a short way and the long way of doing it.

- Half adder -> Full adder-> Ripple-Carry adder.
- Ripple-carry adder + 2's complement -> Signed Addition/Subtraction
- Shift-and-Add -> Unsigned Multiplier
- Sign Logic + Unsigned Multiplier-> Signed Multiplier.
- Signed Multipliers + Signed Adders -> Matrix Multiplication.

6 Behavioral Modeling

In behavioral modeling, the circuit is described using high-level constructs like (if-else) or (case) statement.

7 Architecture

7.1 Matrix Operation

The accelerator computes $C = A * B$ where:

- A is a $3 * 3$ matrix of 16-bit signed integers
- B is a $3 * 1$ matrix of 16-bit signed integers
- C is a $3 * 1$ result matrix of 32-bit signed integers

Mathematical operation:

$$\begin{aligned}C[1,1] &= A[1,1]*B[1,1] + A[1,2]*B[2,1] + A[1,3]*B[3,1] \\C[2,1] &= A[2,1]*B[1,1] + A[2,2]*B[2,1] + A[2,3]*B[3,1] \\C[3,1] &= A[3,1]*B[1,1] + A[3,2]*B[2,1] + A[3,3]*B[3,1]\end{aligned}$$

8 Parameter Configuration(Definitions.vh)

To ensure generality and easy scalability, the system uses parameterized bit-width control declared in Definitions.vh. This file defines the base bit-width control declared in Definitions.vh. This file defines the base bit-width of operands and intermediate result width used during multiplication.

```
'define BIT_WIDTH 16
```

```
'define RESULT_WIDTH (2* BIT_WIDTH)
```

By referencing these constants rather than hard-coding bit-widths in every module, the entire system can be adapted to different precision levels by updating a single definition.

9 Module Descriptions

9.1 Core Arithmetic Modules

1. Half_adder.v

Purpose: Basic half adder building block Inputs:

The half-adder computes the sum and carry of two single bits. It is defined through XOR and AND logic, making it simple yet structurally significant

- a, b (1-bit) Outputs:
- s(sum), c (carry)(1-bit each) Functionality: Implements XOR for sum, AND for carry
- $sum = a \text{ XOR } b$
- $c = a \text{ AND } b$

9.2 project_1.v (Full-Adder)

Although the half-adder can not handle an incoming carry, it is essential for building the Full adder.

Purpose: 1-bit full adder using two half-adders Inputs The full adder adds three bit w: a,b, and carry-in. It is implemented using two half adders and OR gate. This first Half adder produces an intermediate sum and carry. The second Half-adder adds the intermediate sum to the carry-in. The OR of the two carries produces the final carry-out.

- a, b, c_in (1-bit each) Outputs:
- s (sum), c_out (carry out) (1-bit each) Implementation: Uses two Half_adder instances with OR gates for final carry

9.3 sub_controller (Subtraction Logic)

Signed subtraction is implemented using two's complement representation. The Subtraction Controller outputs either the original operand b (for addition) or its two's complement (for subtraction).

This allows a single adder to handle both addition and subtraction without additional logic.

9.4 signed_adder_subtract.v

Purpose: 16-bit signed addition/subtraction unit Parameters:

The ripple-carry adder consists of multiple Full adder Units chained together. Each stage passes its carry-out to the next stage. This design reflects how physical adders in hardware propagate carries.

The 32-bit Ripple-Carry adder supports multiplication accumulation where intermediate results require double the original bit-width

- BIT_WIDTH (default: 16) Inputs:
- a, b (16-bit signed)
- operation (1-bit: 0 =add, 1=subtract) Outputs:
- result (16-bit signed)
- overflow (1-bit overflow flag) Features:
- Two's complement subtraction
- Signed overflow detection
- Operation control via single bit

9.5 ripple_carry_16bit.v

Purpose: 16-bit structural ripple carry adder/subtractor Implementation:

The 16-bit Ripple-carry-add/sub combines the Ripple-Carry adder with the subtraction controller. It also includes overflow detection logic specific to signed arithmetic, checking whether the sign of the output is consistent with the operand expectations.

- Uses 16 instances of project_1 (full adder)
- Generate blocks for scalable instantiation
- Structural implementation as requested features:
- True structural design with discrete components
- Overflow detection for signed arithmetic

10 Matrix Multiplication Modules

Purpose: Complete $3 * 3$ matrix multiplication using structural components
Implementation:

10.1 structural_matrix_multiplication

- 9 Multipliers: One for each $A[i,j] * B[j,1]$ operation
- 6 Adders: Two-stage addition for each result element
- Fully Structural: Uses only custom arithmetic modules Performance: Combinational logic, single clock cycle operation Resource Usage High (9 Multipliers, 6 adders)
- This module instantiates: Three signed_multipliers for each row column combination.
- Two signed_adder_subtractor: modules to accumulate intermediate sums,

11 Testbench Architecture

11.1 signed_adder_subtractor_tb.v

Each arithmetic module includes a dedicated testbench verifying its function prior to integration.

Final verification compares the structural matrix multiplier output to a behavioral reference implementation. The matching results confirm if it is correct.

- Test addition and subtraction operations

- Covers positive/negative combinations
- Includes overflow test cases
- Automated pass/fail reporting
- signed_adder_Subtractor: This module performs signed addition or subtraction based on a control signal. It outputs the computed value along with an overflow flag, determined through sign comparison logic.
- Unsigned Multiplier: The unsigned Multiplier uses the shift-and-add multiplication algorithms. The multiplier bits determine whether partial products are added at each stage, while shifts align contributions by significance.
- Signed Multiplier: First, it determines operand signs, converts both operands to their absolute magnitudes, performs unsigned multiplication, and applies the resulting sign at the end. This is equivalent to hardware multiplication in processors.

11.2 signed_multiplier_tb.v

- Test sign combinations
- Includes edge cases (zero, one, maximum values)
- Compares against Verilog built-in multiplication
- Matrix-typical value testing

11.3 structural_matrix_mult_tb.v

- Complete matrix multiplication testing
- Multiple test vectors with known results
- Matrix display formatting
- Behavioral model comparison

12 Key Design Decisions

12.1 Signed Arithmetic Handling

- Method: Two's complement representation
- Multiplication: Sign extraction then unsigned multiply then sign application
- : Direct signed arithmetic with overflow detection

12.2 16-bit Precision

- Input: 16-bit signed integers(-32,768 to +32,767)
- Output: 32-bit signed integers(prevents overflow)
- Rationale: Sufficient for typical SoC operations

12.3 Structural Implementation

- Full adders: Used project_1 modules as building blocks
- Scalability: Generate blocks for parameterized width.
- Educational: Shows discrete component assembly.

12.4 Error Handling

- Overflow Detection: Implementation in all arithmetic units
- Sign Handling: Robust two's complement operations
- Edge Cases: Proper handling of zeros and maximum values.

13 Performance Characteristics

13.1 Resource Requirements

- Logic elements: 2000-3000(depends of FPGA)
- Multipliers: 9 * 16 bit signed multipliers
- Memory: None
- Clock: Not required

13.2 Timing Characteristics

- Propagation Delay: 50-100ns
- Critical Path: Through Multiplication and Dual addition

14 Conclusion

This project demonstrates how complex arithmetic systems can be constructed from primitive digital logic elements using Verilog. By beginning with half adders and progressing through ripple-carry addition and signed multiplication, the final structural matrix multiplier illustrates how arithmetic datapaths are physically implemented